

Submission Worksheet

Submission Data

Course: IT202-010-S2026

Assignment: IT202 Milestone 2

Student: Ryan C. (rc728)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 802.00/802.00 (100.00%)

Received Grade: 0.00/802.00 (0.00%)

Started: 4/29/2026 12:25:03 AM

Updated: 5/3/2026 7:37:12 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-2/grading/rc728>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-2/view/rc728>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0
- 1. Refer to Milestone2 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWV/wf?glew>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone2 branch
 - 1. `git checkout Milestone2` (ensure proper starting branch)
 - 2. `git pull origin Milestone2` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone2`
 - 4. On Github merge the pull request from Milestone2 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
 - 1. `git checkout qa`
 - 2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/ fields beyond `id`, `created`, `modified`, and respective `is_api` or `api_id` columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an `api_id` or `is_api` column)

id	title	description	picture_url	api_id	created	modified
1	One Piece	Follows the adventures of Monkey D. Luffy, a young boy with the powers of a Devil Fruit that allows him to gain the abilities of a rubber man.	https://onepiece.fandom.com/wiki/One_Piece:_The_Movie:_The_Forest_of_Giants	1	2024-04-29 03:55:00	2024-04-29 03:55:00
2	One Piece: The Movie	The story of the Straw Hat Pirates' first movie adventure, where they go to a tropical island and fight the evil Emperor of the Sea.	https://onepiece.fandom.com/wiki/One_Piece:_The_Movie:_The_Forest_of_Giants	1	2024-04-29 03:55:00	2024-04-29 03:55:00
3	One Piece: The Movie	The story of the Straw Hat Pirates' first movie adventure, where they go to a tropical island and fight the evil Emperor of the Sea.	https://onepiece.fandom.com/wiki/One_Piece:_The_Movie:_The_Forest_of_Giants	1	2024-04-29 03:55:00	2024-04-29 03:55:00
4	One Piece: The Movie	The story of the Straw Hat Pirates' first movie adventure, where they go to a tropical island and fight the evil Emperor of the Sea.	https://onepiece.fandom.com/wiki/One_Piece:_The_Movie:_The_Forest_of_Giants	1	2024-04-29 03:55:00	2024-04-29 03:55:00
5	One Piece: The Movie	The story of the Straw Hat Pirates' first movie adventure, where they go to a tropical island and fight the evil Emperor of the Sea.	https://onepiece.fandom.com/wiki/One_Piece:_The_Movie:_The_Forest_of_Giants	1	2024-04-29 03:55:00	2024-04-29 03:55:00

this shows the db

```

1 CREATE TABLE IF NOT EXISTS anime (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   title VARCHAR(255) NOT NULL,
4   description VARCHAR(255) NOT NULL,
5   picture_url VARCHAR(255),
6   api_id VARCHAR(255),
7   created DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
8   modified DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
9 )

```

this shows the sql



Saved: 4/29/2026 12:49:18 AM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in `.sql`)

URL#1

[https://github.com/524RC/RC728-](https://github.com/524RC/RC728-IT2026prod/public_html/project/sql/006_create_table_anime.sql)
[IT2026prod/public_html/project/sql/006_create_table_anime.sql](https://github.com/524RC/RC728-IT2026prod/public_html/project/sql/006_create_table_anime.sql)


URL

<https://github.com/524RC/RC728>


URL#2

<https://github.com/524RC/RC728->



<https://github.com/524RC/RC728>



IT2021000prod/public_html/project/sql/007_insert_anime_samples.sql



Saved: 4/29/2026 12:49:18 AM

⇒ Part 3:

Progress: 100%

Details:

- Briefly note what each field/ property represents

Your Response:

id - the unique identifier for each anime record which is also the primary key
mal_id - the ID of the anime from MyAnimeList API
is_api - is checks if the record was fetched from the api
title - is the name of the anime
description - a short summary of the anime
picture_url - its the link to the anime's image
mal_url - the link to the anime's page on MyAnimeList
created - a timestamp of when the record was created
modified - a timestamp of when the record was last updated



Saved: 4/29/2026 12:49:18 AM

≡ Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

📄 Part 1:

Progress: 100%

Details:

- Show the code that handles the data transformation/ conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB

```
const axios = require('axios');
const { db } = require('../db');
const { Anime } = require('../models');

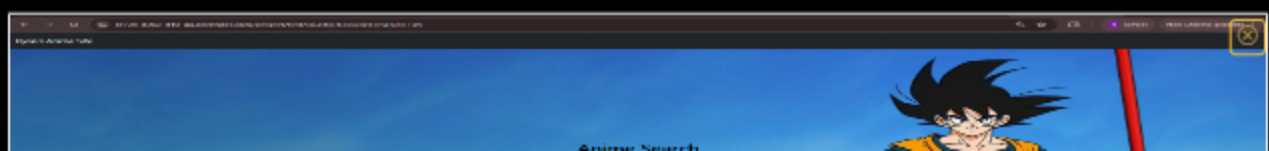
// Fetch anime data from MyAnimeList API
const fetchAnimeData = async () => {
  const url = 'https://api.myanime.com/v1/anime';
  const response = await axios.get(url);
  const animeList = response.data;

  // Transform API response to match database schema
  const transformedData = animeList.map(anime => {
    id: anime.mal_id,
    title: anime.title,
    description: anime.description,
    picture_url: anime.picture_url,
    mal_url: `https://myanimelist.net/anime/${anime.mal_id}`,
    created: new Date(),
    modified: new Date(),
  });

  // Insert transformed data into database
  Anime.bulkCreate(transformedData, {
    validate: true,
    update: true,
  });
};

// Run the fetch function
fetchAnimeData();
```

this shows the code





this shows render.com

#	Q	id	mat_id	* %_apt	* title	description	picture_url	mat_url	* created	* modified
		id	id	score(%)	varchar(255)	text	varchar(255)	varchar(255)	datetime	datetime
10	>	1	21	1	One Piece	Barely surviving in a barrel :	https://cdn.myanimelist.net/	https://myanimelist.net/ani	2020-04-29 09:55:06	2020-04-29 09:55:06
10	>	2	30276	1	One Punch Man	The seemingly unimpressive	https://cdn.myanimelist.net/	https://myanimelist.net/ani	2020-04-29 09:55:06	2020-04-29 09:55:06
10	>	3	32182	1	Mob Psycho 100	Eightth-grader Shigeo "Mob"	https://cdn.myanimelist.net/	https://myanimelist.net/ani	2020-04-29 09:55:06	2020-04-29 09:55:06
10	>	4	(N/A)	0	My Favorite Underdog Ans	A personal recommendation (N/A)	(N/A)	(N/A)	2020-04-29 09:55:06	2020-04-29 09:55:06
10	>	5	(N/A)	0	Local Animation Project	(N/A)	(N/A)	(N/A)	2020-04-29 09:55:06	2020-04-29 09:55:06
10	>	11	(N/A)	1	One Piece Film Z	The "Stops" that makes enter:	https://cdn.myanimelist.net/	(N/A)	2020-04-29 09:55:06	2020-04-29 09:55:06
10	>	12	1726	1	Naruto: Shippuden	It had been two and a half y:	https://cdn.myanimelist.net/	https://myanimelist.net/ani	2020-04-29 09:55:06	2020-04-29 09:55:06

this shows the db

 Saved: 5/3/2026 4:56:57 AM

Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

URL#1

<https://rc/28-it202-010-prod.onrender.com/project/testApi.php?keyword=Naruto>

<https://rc728-it202-010-prod.onre>

URL#2

<https://github.com/524RC/RC728-IT20240102>

<https://github.com/524RC/RC728>

 Saved: 5/3/2026 4:56:57 AM

≡ Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/ extraction logic to get the API data to the shape your application intends to use

Your Response:

I am using a subset of the API data by only using relevant fields like id, title, description, and the URLs. Some of the data are also converted to match the database structure like myanimelist_id is changed to mal_id and myanimelist_url is changed to mal_url. I also have a custom field is_api and set it to 1 to indicate that the data was fetched from the API. The transformation/ extraction logic works by decoding the JSON response and looping through each anime result and extracting the required fields and then inserting them into the Anime table. This is done to make sure that the API data is stored in a consistent format that matches the structure that the application uses.



Saved: 5/3/2026 4:56:57 AM

⇒ Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

Duplicates are handled by using ON DUPLICATE KEY UPDATE based on the unique mal_id field. If the same anime is fetched again from the API and it already exists in the database without any changes, then the record remains the same. If the API data has changed, then the existing record is updated with the new values instead of inserting a duplicate.



Saved: 4/29/2026 2:01:01 AM

⇒ Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/ awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/ missing content)

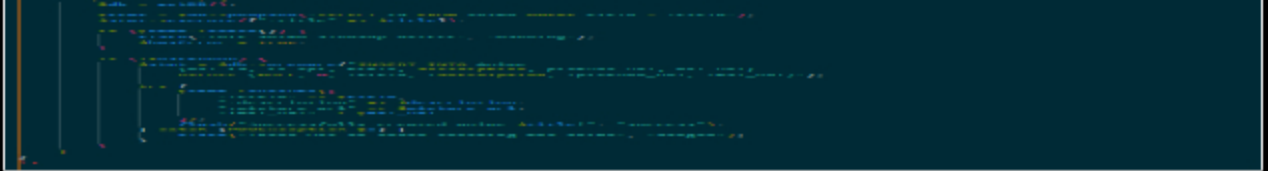
Your Response:

 Saved: 4/29/2026 2:03:36 AM

Progress: 100%

Progress: 100%

Progress: 100%



This shows the php validation



Saved: 5/3/2026 1:55:05 AM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic
- Note which role(s) have access to this create form

Your Response:

HTML: The form uses the built in validation attributes like required, maxlength = "255", and type = "url". I used the required attribute to ensure that the title field must be filled before submission. The maxlength allows for a limit of the title being 255 characters long. Finally the type= "url" allows that the upicture url and the MyAnimeList url use a valid url format.

js: The JavaScript validation runs when the form is submitted by using the validate function where it checks that the title is not empty and does not exceed 255 characters. along with that it also checks to see if the picture url and the MyAnimeList url start wiht http to make sure that it is a valid link. If any of these checks fail, then a alert message pops up and the submission is cancelled.

PHP: The php validation works by running after the form is submitted and checks if the title is empty or if it exceeds 255 characters. It also uses filter_var() to check if the url is valid. To prevent duplicate animes being in the database, it checks if the anime already exists in the database. If any of these validations fail, an error message is displayed.

role access: the only role that has access to this from si the admin role which the form checks for by using has_role("Admin")on the top of the page which checks if the user has the admin role or not and prevents non-admin users from accessing the form.



Saved: 5/3/2026 1:55:05 AM

📁 Task #2 (60 pts.) - Examples

Progress: 100%

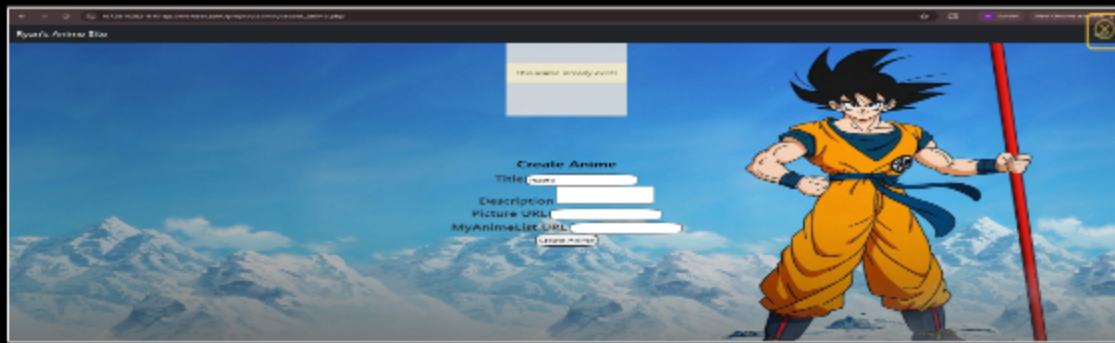
Details:

Show a successful creation

- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior occurs)
- Demonstrate various validation errors
- Ensure Render.com qa url is visible



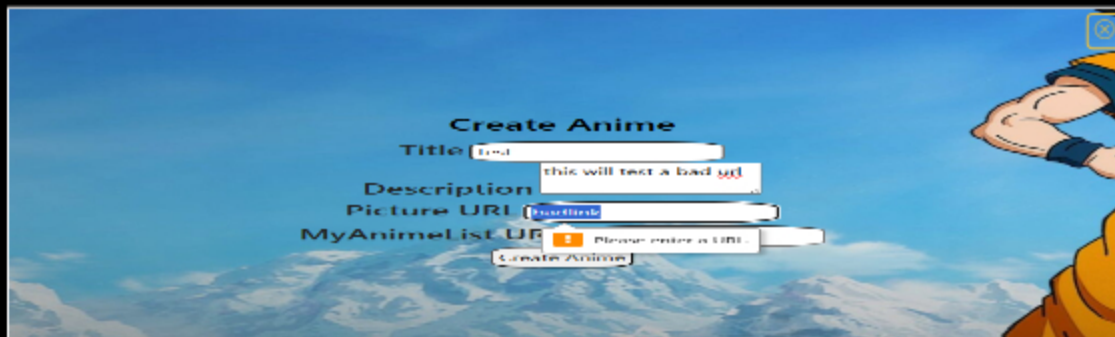
this shows a successful creation



this shows an attempt at duplication



this shows an empty title error

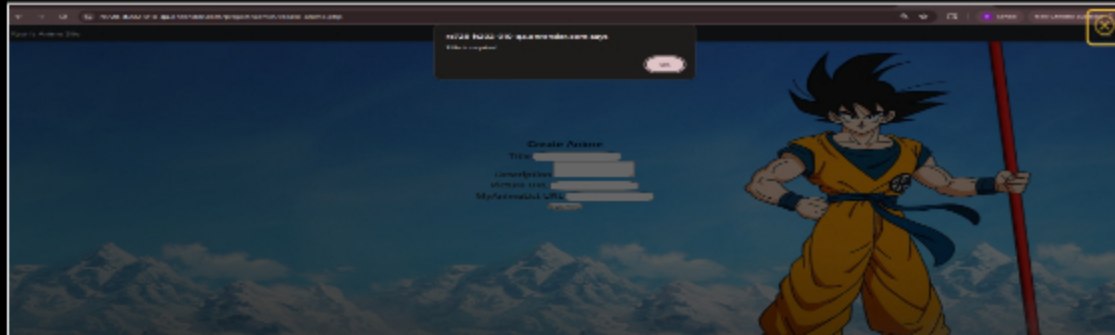


this shows an invalid url link error

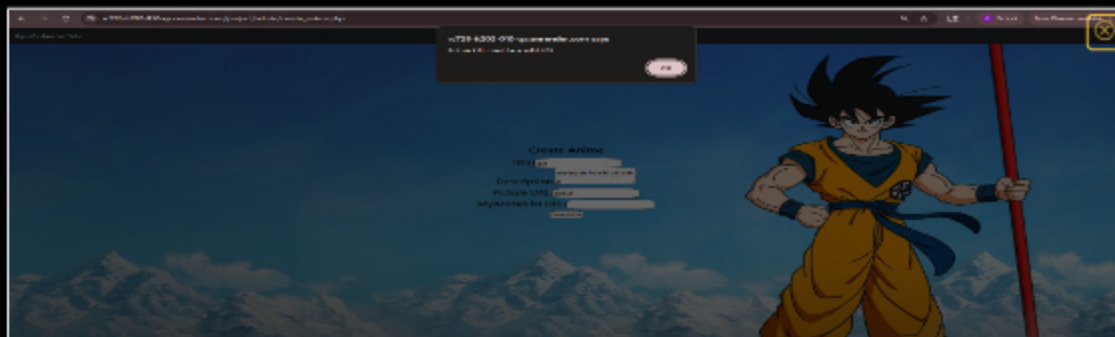




this shows that the anime was created and was added to the db



this shows js empty title error



this shows js invalid url link error



Saved: 5/3/2026 4:55:36 AM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL#1

https://rc728-it202-010-prod.onrender.com/project/admin/create_anime.php



URL

https://rc728-it202-010-prod.onrender.com/project/admin/create_anime.php



URL#2

<https://github.com/524RC/RC728-IT202-010>



URL

<https://github.com/524RC/RC728-IT202-010>





Section #3: (150 pts.) Data List Page (Many Items)

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the filter/ sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/ grid output
- Styling must be applied

```

10 <script>
11     $(document).ready(function() {
12         // Handle the filter/sort logic
13         if (typeof $('#filter').val() != '') {
14             $('#filter').val().split(' ').forEach(function(word) {
15                 $('#filter').val().replace(word, '<span>[X]</span>');
16             });
17         }
18         // Handle the record limit
19         if (typeof $('#limit').val() != '') {
20             $('#limit').val().split(' ').forEach(function(word) {
21                 $('#limit').val().replace(word, '<span>[X]</span>');
22             });
23         }
24         // Handle the sort logic
25         if (typeof $('#sort').val() != '') {
26             $('#sort').val().split(' ').forEach(function(word) {
27                 $('#sort').val().replace(word, '<span>[X]</span>');
28             });
29         }
30         // Handle the pagination
31         if (typeof $('#page').val() != '') {
32             $('#page').val().split(' ').forEach(function(word) {
33                 $('#page').val().replace(word, '<span>[X]</span>');
34             });
35         }
36         // Handle the search
37         if (typeof $('#search').val() != '') {
38             $('#search').val().split(' ').forEach(function(word) {
39                 $('#search').val().replace(word, '<span>[X]</span>');
40             });
41         }
42         // Handle the filter/sort logic
43         if (typeof $('#filter').val() != '') {
44             $('#filter').val().split(' ').forEach(function(word) {
45                 $('#filter').val().replace(word, '<span>[X]</span>');
46             });
47         }
48         // Handle the record limit
49         if (typeof $('#limit').val() != '') {
50             $('#limit').val().split(' ').forEach(function(word) {
51                 $('#limit').val().replace(word, '<span>[X]</span>');
52             });
53         }
54         // Handle the sort logic
55         if (typeof $('#sort').val() != '') {
56             $('#sort').val().split(' ').forEach(function(word) {
57                 $('#sort').val().replace(word, '<span>[X]</span>');
58             });
59         }
60         // Handle the pagination
61         if (typeof $('#page').val() != '') {
62             $('#page').val().split(' ').forEach(function(word) {
63                 $('#page').val().replace(word, '<span>[X]</span>');
64             });
65         }
66         // Handle the search
67         if (typeof $('#search').val() != '') {
68             $('#search').val().split(' ').forEach(function(word) {
69                 $('#search').val().replace(word, '<span>[X]</span>');
70             });
71         }
72     });
73 }

```

this shows the entire filter/sort logic

```

1 if (isset($_GET['limit'])) {
2     $limit = (int)se($_GET, 'limit', 10, false);
3     if ($limit < 1 || $limit > 100) {
4         $limit = 10;
5     }
6 }

```

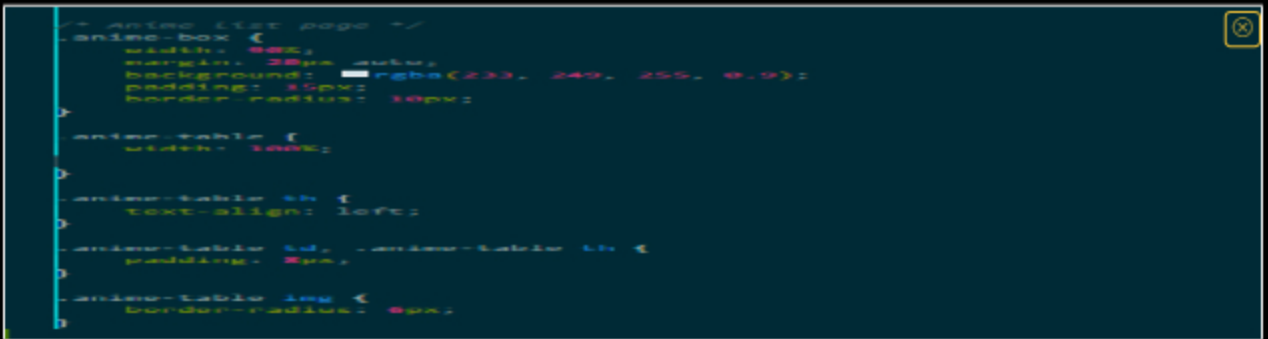
this shows the code snippet of the record limit

```

1 <table>
2 <thead>
3 <tr>
4 <th>ID</th>
5 <th>Name</th>
6 <th>Email</th>
7 <th>Phone</th>
8 <th>Address</th>
9 </tr>
10 </thead>
11 <tbody>
12 <tr>
13 <td>1</td>
14 <td>John</td>
15 <td>john.doe@example.com</td>
16 <td>1234567890</td>
17 <td>123 Main St</td>
18 </tr>
19 <tr>
20 <td>2</td>
21 <td>Jane</td>
22 <td>jane.doe@example.com</td>
23 <td>9876543210</td>
24 <td>456 Main St</td>
25 </tr>
26 <tr>
27 <td>3</td>
28 <td>Bob</td>
29 <td>bob.doe@example.com</td>
30 <td>5555555555</td>
31 <td>789 Main St</td>
32 </tr>
33 <tr>
34 <td>4</td>
35 <td>Alice</td>
36 <td>alice.doe@example.com</td>
37 <td>1111111111</td>
38 <td>101 Main St</td>
39 </tr>
40 <tr>
41 <td>5</td>
42 <td>Charlie</td>
43 <td>charlie.doe@example.com</td>
44 <td>2222222222</td>
45 <td>202 Main St</td>
46 </tr>
47 <tr>
48 <td>6</td>
49 <td>Diana</td>
50 <td>diana.doe@example.com</td>
51 <td>3333333333</td>
52 <td>303 Main St</td>
53 </tr>
54 <tr>
55 <td>7</td>
56 <td>Ethan</td>
57 <td>ethan.doe@example.com</td>
58 <td>4444444444</td>
59 <td>404 Main St</td>
60 </tr>
61 <tr>
62 <td>8</td>
63 <td>Fiona</td>
64 <td>fiona.doe@example.com</td>
65 <td>5555555555</td>
66 <td>505 Main St</td>
67 </tr>
68 <tr>
69 <td>9</td>
70 <td>George</td>
71 <td>george.doe@example.com</td>
72 <td>6666666666</td>
73 <td>606 Main St</td>
74 </tr>
75 <tr>
76 <td>10</td>
77 <td>Hannah</td>
78 <td>hannah.doe@example.com</td>
79 <td>7777777777</td>
80 <td>707 Main St</td>
81 </tr>
82 <tr>
83 <td>11</td>
84 <td>Ian</td>
85 <td>ian.doe@example.com</td>
86 <td>8888888888</td>
87 <td>808 Main St</td>
88 </tr>
89 <tr>
90 <td>12</td>
91 <td>Julia</td>
92 <td>julia.doe@example.com</td>
93 <td>9999999999</td>
94 <td>909 Main St</td>
95 </tr>
96 <tr>
97 <td>13</td>
98 <td>Kevin</td>
99 <td>kevin.doe@example.com</td>
100 <td>1010101010</td>
101 <td>1010 Main St</td>
102 </tr>
103 <tr>
104 <td>14</td>
105 <td>Liam</td>
106 <td>liam.doe@example.com</td>
107 <td>2020202020</td>
108 <td>2020 Main St</td>
109 </tr>
110 <tr>
111 <td>15</td>
112 <td>Mia</td>
113 <td>mia.doe@example.com</td>
114 <td>3030303030</td>
115 <td>3030 Main St</td>
116 </tr>
117 <tr>
118 <td>16</td>
119 <td>Noah</td>
120 <td>noah.doe@example.com</td>
121 <td>4040404040</td>
122 <td>4040 Main St</td>
123 </tr>
124 <tr>
125 <td>17</td>
126 <td>Olivia</td>
127 <td>olivia.doe@example.com</td>
128 <td>5050505050</td>
129 <td>5050 Main St</td>
130 </tr>
131 <tr>
132 <td>18</td>
133 <td>Peter</td>
134 <td>peter.doe@example.com</td>
135 <td>6060606060</td>
136 <td>6060 Main St</td>
137 </tr>
138 <tr>
139 <td>19</td>
140 <td>Quinn</td>
141 <td>quinn.doe@example.com</td>
142 <td>7070707070</td>
143 <td>7070 Main St</td>
144 </tr>
145 <tr>
146 <td>20</td>
147 <td>Ryan</td>
148 <td>ryan.doe@example.com</td>
149 <td>8080808080</td>
150 <td>8080 Main St</td>
151 </tr>
152 <tr>
153 <td>21</td>
154 <td>Sophia</td>
155 <td>sophia.doe@example.com</td>
156 <td>9090909090</td>
157 <td>9090 Main St</td>
158 </tr>
159 <tr>
160 <td>22</td>
161 <td>Theodore</td>
162 <td>theodore.doe@example.com</td>
163 <td>1010101010</td>
164 <td>1010 Main St</td>
165 </tr>
166 <tr>
167 <td>23</td>
168 <td>Uma</td>
169 <td>uma.doe@example.com</td>
170 <td>2020202020</td>
171 <td>2020 Main St</td>
172 </tr>
173 <tr>
174 <td>24</td>
175 <td>Victor</td>
176 <td>victor.doe@example.com</td>
177 <td>3030303030</td>
178 <td>3030 Main St</td>
179 </tr>
180 <tr>
181 <td>25</td>
182 <td>Wendy</td>
183 <td>wendy.doe@example.com</td>
184 <td>4040404040</td>
185 <td>4040 Main St</td>
186 </tr>
187 <tr>
188 <td>26</td>
189 <td>Xavier</td>
190 <td>xavier.doe@example.com</td>
191 <td>5050505050</td>
192 <td>5050 Main St</td>
193 </tr>
194 <tr>
195 <td>27</td>
196 <td>Yara</td>
197 <td>yara.doe@example.com</td>
198 <td>6060606060</td>
199 <td>6060 Main St</td>
200 </tr>
201 <tr>
202 <td>28</td>
203 <td>Zoe</td>
204 <td>zoe.doe@example.com</td>
205 <td>7070707070</td>
206 <td>7070 Main St</td>
207 </tr>
208 <tr>
209 <td>29</td>
210 <td>Adam</td>
211 <td>adam.doe@example.com</td>
212 <td>8080808080</td>
213 <td>8080 Main St</td>
214 </tr>
215 <tr>
216 <td>30</td>
217 <td>Bella</td>
218 <td>bella.doe@example.com</td>
219 <td>9090909090</td>
220 <td>9090 Main St</td>
221 </tr>
222 <tr>
223 <td>31</td>
224 <td>Caleb</td>
225 <td>caleb.doe@example.com</td>
226 <td>1010101010</td>
227 <td>1010 Main St</td>
228 </tr>
229 <tr>
230 <td>32</td>
231 <td>Dora</td>
232 <td>dora.doe@example.com</td>
233 <td>2020202020</td>
234 <td>2020 Main St</td>
235 </tr>
236 <tr>
237 <td>33</td>
238 <td>Eli</td>
239 <td>eli.doe@example.com</td>
240 <td>3030303030</td>
241 <td>3030 Main St</td>
242 </tr>
243 <tr>
244 <td>34</td>
245 <td>Fiona</td>
246 <td>fiona.doe@example.com</td>
247 <td>4040404040</td>
248 <td>4040 Main St</td>
249 </tr>
250 <tr>
251 <td>35</td>
252 <td>Gavin</td>
253 <td>gavin.doe@example.com</td>
254 <td>5050505050</td>
255 <td>5050 Main St</td>
256 </tr>
257 <tr>
258 <td>36</td>
259 <td>Hannah</td>
260 <td>hannah.doe@example.com</td>
261 <td>6060606060</td>
262 <td>6060 Main St</td>
263 </tr>
264 <tr>
265 <td>37</td>
266 <td>Ivan</td>
267 <td>ivan.doe@example.com</td>
268 <td>7070707070</td>
269 <td>7070 Main St</td>
270 </tr>
271 <tr>
272 <td>38</td>
273 <td>Julia</td>
274 <td>julia.doe@example.com</td>
275 <td>8080808080</td>
276 <td>8080 Main St</td>
277 </tr>
278 <tr>
279 <td>39</td>
280 <td>Kevin</td>
281 <td>kevin.doe@example.com</td>
282 <td>9090909090</td>
283 <td>9090 Main St</td>
284 </tr>
285 <tr>
286 <td>40</td>
287 <td>Liam</td>
288 <td>liam.doe@example.com</td>
289 <td>1010101010</td>
290 <td>1010 Main St</td>
291 </tr>
292 <tr>
293 <td>41</td>
294 <td>Mia</td>
295 <td>mia.doe@example.com</td>
296 <td>2020202020</td>
297 <td>2020 Main St</td>
298 </tr>
299 <tr>
300 <td>42</td>
301 <td>Noah</td>
302 <td>noah.doe@example.com</td>
303 <td>3030303030</td>
304 <td>3030 Main St</td>
305 </tr>
306 <tr>
307 <td>43</td>
308 <td>Olivia</td>
309 <td>olivia.doe@example.com</td>
310 <td>4040404040</td>
311 <td>4040 Main St</td>
312 </tr>
313 <tr>
314 <td>44</td>
315 <td>Peter</td>
316 <td>peter.doe@example.com</td>
317 <td>5050505050</td>
318 <td>5050 Main St</td>
319 </tr>
320 <tr>
321 <td>45</td>
322 <td>Quinn</td>
323 <td>quinn.doe@example.com</td>
324 <td>6060606060</td>
325 <td>6060 Main St</td>
326 </tr>
327 <tr>
328 <td>46</td>
329 <td>Ryan</td>
330 <td>ryan.doe@example.com</td>
331 <td>7070707070</td>
332 <td>7070 Main St</td>
333 </tr>
334 <tr>
335 <td>47</td>
336 <td>Sophia</td>
337 <td>sophia.doe@example.com</td>
338 <td>8080808080</td>
339 <td>8080 Main St</td>
340 </tr>
341 <tr>
342 <td>48</td>
343 <td>Theodore</td>
344 <td>theodore.doe@example.com</td>
345 <td>9090909090</td>
346 <td>9090 Main St</td>
347 </tr>
348 <tr>
349 <td>49</td>
350 <td>Uma</td>
351 <td>uma.doe@example.com</td>
352 <td>1010101010</td>
353 <td>1010 Main St</td>
354 </tr>
355 <tr>
356 <td>50</td>
357 <td>Victor</td>
358 <td>victor.doe@example.com</td>
359 <td>2020202020</td>
360 <td>2020 Main St</td>
361 </tr>
362 <tr>
363 <td>51</td>
364 <td>Wendy</td>
365 <td>wendy.doe@example.com</td>
366 <td>3030303030</td>
367 <td>3030 Main St</td>
368 </tr>
369 <tr>
370 <td>52</td>
371 <td>Xavier</td>
372 <td>xavier.doe@example.com</td>
373 <td>4040404040</td>
374 <td>4040 Main St</td>
375 </tr>
376 <tr>
377 <td>53</td>
378 <td>Yara</td>
379 <td>yara.doe@example.com</td>
380 <td>5050505050</td>
381 <td>5050 Main St</td>
382 </tr>
383 <tr>
384 <td>54</td>
385 <td>Zoe</td>
386 <td>zoe.doe@example.com</td>
387 <td>6060606060</td>
388 <td>6060 Main St</td>
389 </tr>
390 <tr>
391 <td>55</td>
392 <td>Adam</td>
393 <td>adam.doe@example.com</td>
394 <td>7070707070</td>
395 <td>7070 Main St</td>
396 </tr>
397 <tr>
398 <td>56</td>
399 <td>Bella</td>
400 <td>bella.doe@example.com</td>
401 <td>8080808080</td>
402 <td>8080 Main St</td>
403 </tr>
404 <tr>
405 <td>57</td>
406 <td>Caleb</td>
407 <td>caleb.doe@example.com</td>
408 <td>9090909090</td>
409 <td>9090 Main St</td>
410 </tr>
411 <tr>
412 <td>58</td>
413 <td>Dora</td>
414 <td>dora.doe@example.com</td>
415 <td>1010101010</td>
416 <td>1010 Main St</td>
417 </tr>
418 <tr>
419 <td>59</td>
420 <td>Eli</td>
421 <td>eli.doe@example.com</td>
422 <td>2020202020</td>
423 <td>2020 Main St</td>
424 </tr>
425 <tr>
426 <td>60</td>
427 <td>Fiona</td>
428 <td>fiona.doe@example.com</td>
429 <td>3030303030</td>
430 <td>3030 Main St</td>
431 </tr>
432 <tr>
433 <td>61</td>
434 <td>Gavin</td>
435 <td>gavin.doe@example.com</td>
436 <td>4040404040</td>
437 <td>4040 Main St</td>
438 </tr>
439 <tr>
440 <td>62</td>
441 <td>Hannah</td>
442 <td>hannah.doe@example.com</td>
443 <td>5050505050</td>
444 <td>5050 Main St</td>
445 </tr>
446 <tr>
447 <td>63</td>
448 <td>Ivan</td>
449 <td>ivan.doe@example.com</td>
450 <td>6060606060</td>
451 <td>6060 Main St</td>
452 </tr>
453 <tr>
454 <td>64</td>
455 <td>Julia</td>
456 <td>julia.doe@example.com</td>
457 <td>7070707070</td>
458 <td>7070 Main St</td>
459 </tr>
460 <tr>
461 <td>65</td>
462 <td>Kevin</td>
463 <td>kevin.doe@example.com</td>
464 <td>8080808080</td>
465 <td>8080 Main St</td>
466 </tr>
467 <tr>
468 <td>66</td>
469 <td>Liam</td>
470 <td>liam.doe@example.com</td>
471 <td>9090909090</td>
472 <td>9090 Main St</td>
473 </tr>
474 <tr>
475 <td>67</td>
476 <td>Mia</td>
477 <td>mia.doe@example.com</td>
478 <td>1010101010</td>
479 <td>1010 Main St</td>
480 </tr>
481 <tr>
482 <td>68</td>
483 <td>Noah</td>
484 <td>noah.doe@example.com</td>
485 <td>2020202020</td>
486 <td>2020 Main St</td>
487 </tr>
488 <tr>
489 <td>69</td>
490 <td>Olivia</td>
491 <td>olivia.doe@example.com</td>
492 <td>3030303030</td>
493 <td>3030 Main St</td>
494 </tr>
495 <tr>
496 <td>70</td>
497 <td>Peter</td>
498 <td>peter.doe@example.com</td>
499 <td>4040404040</td>
500 <td>4040 Main St</td>
501 </tr>
502 <tr>
503 <td>71</td>
504 <td>Quinn</td>
505 <td>quinn.doe@example.com</td>
506 <td>5050505050</td>
507 <td>5050 Main St</td>
508 </tr>
509 <tr>
510 <td>72</td>
511 <td>Ryan</td>
512 <td>ryan.doe@example.com</td>
513 <td>6060606060</td>
514 <td>6060 Main St</td>
515 </tr>
516 <tr>
517 <td>73</td>
518 <td>Sophia</td>
519 <td>sophia.doe@example.com</td>
520 <td>7070707070</td>
521 <td>7070 Main St</td>
522 </tr>
523 <tr>
524 <td>74</td>
525 <td>Theodore</td>
526 <td>theodore.doe@example.com</td>
527 <td>8080808080</td>
528 <td>8080 Main St</td>
529 </tr>
530 <tr>
531 <td>75</td>
532 <td>Uma</td>
533 <td>uma.doe@example.com</td>
534 <td>9090909090</td>
535 <td>9090 Main St</td>
536 </tr>
537 <tr>
538 <td>76</td>
539 <td>Victor</td>
540 <td>victor.doe@example.com</td>
541 <td>1010101010</td>
542 <td>1010 Main St</td>
543 </tr>
544 <tr>
545 <td>77</td>
546 <td>Wendy</td>
547 <td>wendy.doe@example.com</td>
548 <td>2020202020</td>
549 <td>2020 Main St</td>
550 </tr>
551 <tr>
552 <td>78</td>
553 <td>Xavier</td>
554 <td>xavier.doe@example.com</td>
555 <td>3030303030</td>
556 <td>3030 Main St</td>
557 </tr>
558 <tr>
559 <td>79</td>
560 <td>Yara</td>
561 <td>yara.doe@example.com</td>
562 <td>4040404040</td>
563 <td>4040 Main St</td>
564 </tr>
565 <tr>
566 <td>80</td>
567 <td>Zoe</td>
568 <td>zoe.doe@example.com</td>
569 <td>5050505050</td>
570 <td>5050 Main St</td>
571 </tr>
572 <tr>
573 <td>81</td>
574 <td>Adam</td>
575 <td>adam.doe@example.com</td>
576 <td>6060606060</td>
577 <td>6060 Main St</td>
578 </tr>
579 <tr>
580 <td>82</td>
581 <td>Bella</td>
582 <td>bella.doe@example.com</td>
583 <td>7070707070</td>
584 <td>7070 Main St</td>
585 </tr>
586 <tr>
587 <td>83</td>
588 <td>Caleb</td>
589 <td>caleb.doe@example.com</td>
590 <td>8080808080</td>
591 <td>8080 Main St</td>
592 </tr>
593 <tr>
594 <td>84</td>
595 <td>Dora</td>
596 <td>dora.doe@example.com</td>
597 <td>9090909090</td>
598 <td>9090 Main St</td>
599 </tr>
600 <tr>
601 <td>85</td>
602 <td>Eli</td>
603 <td>eli.doe@example.com</td>
604 <td>1010101010</td>
605 <td>1010 Main St</td>
606 </tr>
607 <tr>
608 <td>86</td>
609 <td>Fiona</td>
610 <td>fiona.doe@example.com</td>
611 <td>2020202020</td>
612 <td>2020 Main St</td>
613 </tr>
614 <tr>
615 <td>87</td>
616 <td>Gavin</td>
617 <td>gavin.doe@example.com</td>
618 <td>3030303030</td>
619 <td>3030 Main St</td>
620 </tr>
621 <tr>
622 <td>88</td>
623 <td>Hannah</td>
624 <td>hannah.doe@example.com</td>
625 <td>4040404040</td>
626 <td>4040 Main St</td>
627 </tr>
628 <tr>
629 <td>89</td>
630 <td>Ivan</td>
631 <td>ivan.doe@example.com</td>
632 <td>5050505050</td>
633 <td>5050 Main St</td>
634 </tr>
635 <tr>
636 <td>90</td>
637 <td>Julia</td>
638 <td>julia.doe
```




this shows the list output logic



This shows the css for the styling done to the list



Saved: 5/3/2026 3:04:36 AM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how the filter/ sort logic works and formulates the results
- Briefly explain how the limit is validated and applied server-side
- Briefly explain how the list/ grid output is generated
- Briefly note the styling choices

Your Response:

Filter/ sort logic: the filter/ sort logic works by using the `$_GET` to be able to modify the SQL query. If an item for search is provided, then it added a `WHERE title LIKE` clause to filter the results. `$allowed_sorts = ["title", "created", "modified", "is_api"]`; is used to prevent any invalid input and then applied using the `ORDER BY` which it organizes the results either alphabetically or by how recent.

Limit validation: The limit validation works by retrieving the limit from `$_GET` and converted to an integer. Then it is validated to check if it falls within the given range of 1 to 100. If the value is outside the range, then it defaults to 10. this allows for the query to not returns too many records at once.

list/ grid output: the list is generated through a loop system where it loops through the query result using a `foreach` loop. Each record is displayed as a whole row in the HTML table with it showing the fields like ID, image, title, description, and source. Then there are links to bring the user into view, edit, or delete the anime which is included for each anime.

Styling: I styled it using CSS to make it more readable because the background made it a little hard to see the text which was in black. I used a semi-transparent background on the main container so that the background image is still seen but not too much so the text is still readable. I added padding and spacing for clarity on which category each text belonged to and finally each

image has a rounded corner to a nicer, less rough look to it. The table is also set to full width which allows for the alignment of the content.



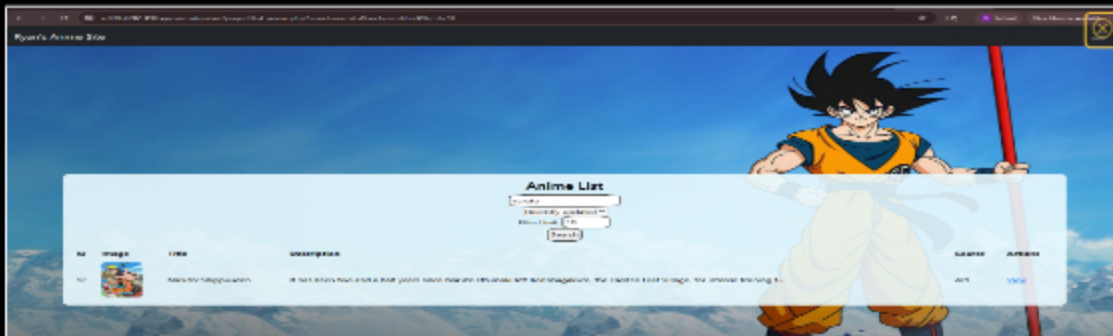
Saved: 5/3/2026 3:04:36 AM

Task #2 (60 pts.) - Examples

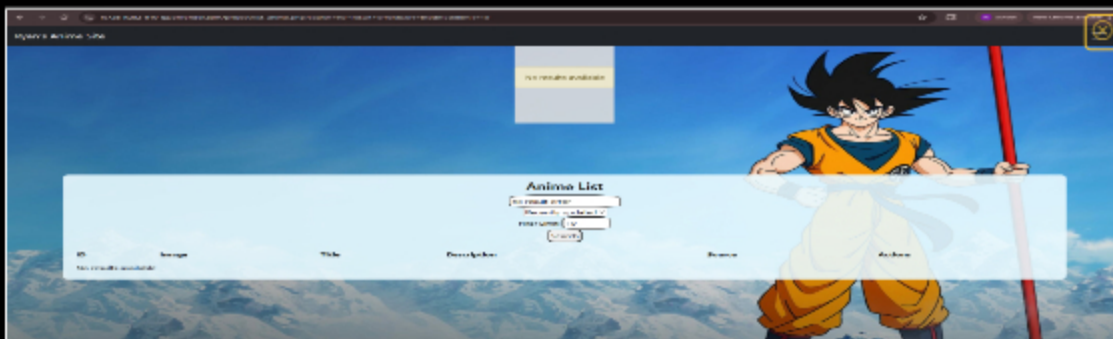
Progress: 100%

Details:

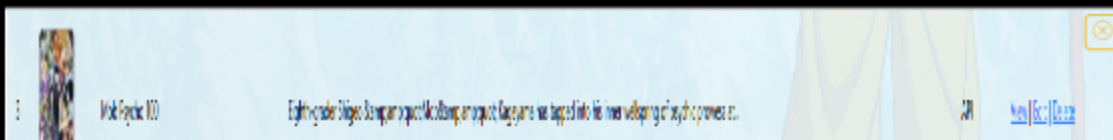
- Show a few variations of filter/ sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com qa url is visible

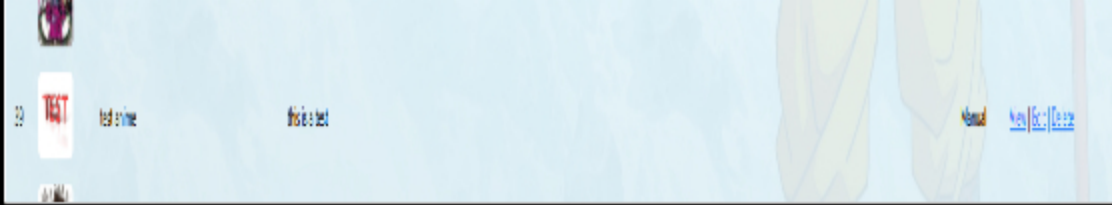


This shows the filter working



this shows no result found error

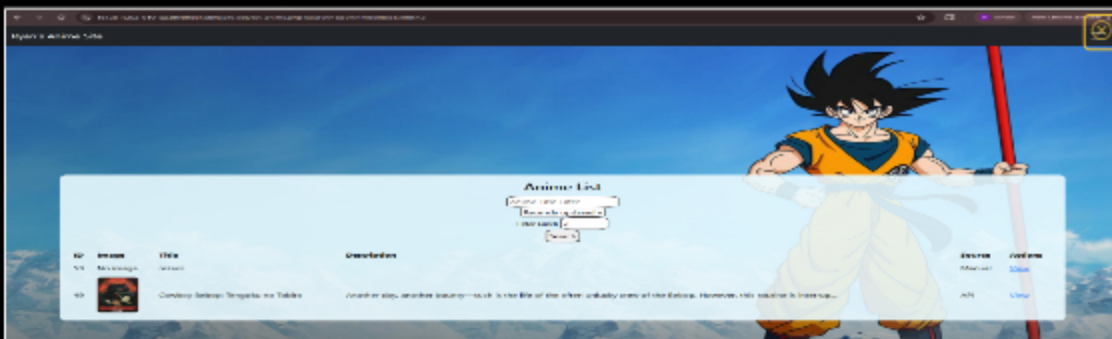




this shows two different values one with manual and one with api



this shows the limit being 5



this shows the limit being 2



this shows what the admin can see with edit and delete being there



Saved: 5/3/2026 3:17:51 AM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page

- Include pull request link for this feature

URL#1

https://rc728-it202-010-prod.onrender.com/project/list_anime.php



URL

https://rc728-it202-010-prod.onrender.com/project/list_anime.php



URL#2

<https://github.com/524RC/RC728-IT202-010>



URL

<https://github.com/524RC/RC728-IT202-010>



Saved: 5/3/2026 3:18:14 AM

Section #4: (150 pts.) View Details Page (Single Item)

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect back to the previous page with an appropriate message)
- Show the code that generates the entity output
- Styling must be applied

```
$id = (int)se($_GET, "id", 0, false);

if ($id <= 0) {
    flash("Invalid anime id", "warning");
    die(header("Location: " . get_url("list_anime.php")));
}
```

this shows the code for ID loading and invalid id handling

```
$db = getDB();
$stmt = $db->prepare("SELECT * FROM Anime WHERE id = :id");
$stmt->execute(["id" => $id]);
$anime = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$anime) {
    flash("Anime not found", "warning");
    die(header("Location: " . get_url("list_anime.php")));
}
```

This shows the code for database fetch and id not found

This shows the code for the output

this shows the css used for style which is the same for the list

≡ Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

Your Response:

loading logic: the loading logic works by grabbing the id from the url using `$_GET` and converting it into an integer. If this id is invalid, then the user is redirected back to the list page with an error message and looks through the database for an anime with that ID. if there is no anime with that ID, it will redirect back again with a "not found" message.

entity output: for the entity output i generated it by using php inside html to show the anime's data. All the title, the image, description, and the source are all printed using the se() function to show the values. There are also conditonal checks so that the image and MyAnimeList links only appear if they actually exist. Also admins get to see the edit and delete feature which non-admin users cannot see.

styling: The styling i did is similar to the rest where it uses a transparent background for

styling. The styling I did is similar to the rest where it uses a transparent background for readability and padding and spacing to separate sections more clearly. I also included rounded corners on the images to give it a cleaner look.



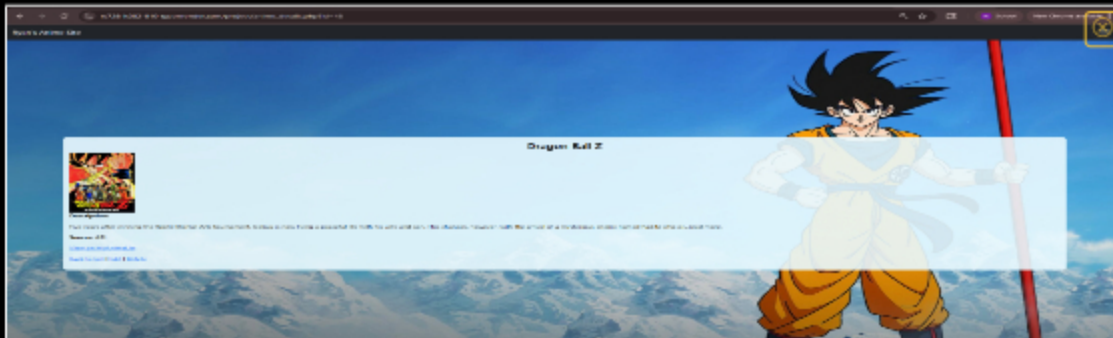
Saved: 5/3/2026 4:37:23 AM

Task #2 (60 pts.) - Examples

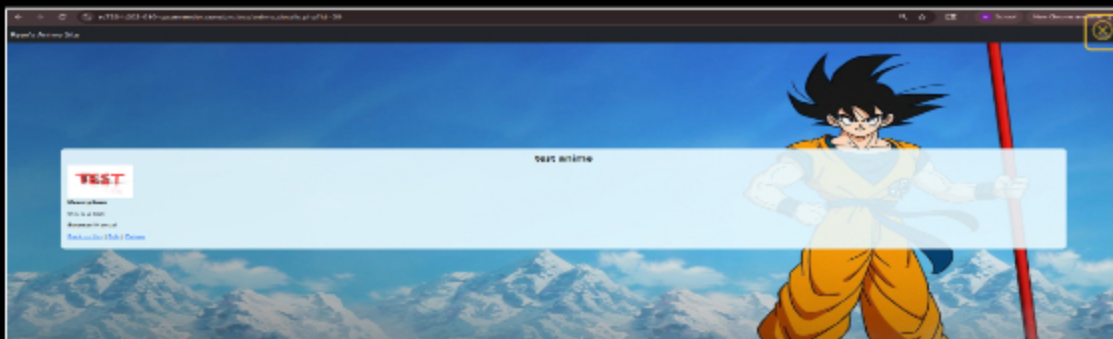
Progress: 100%

Details:

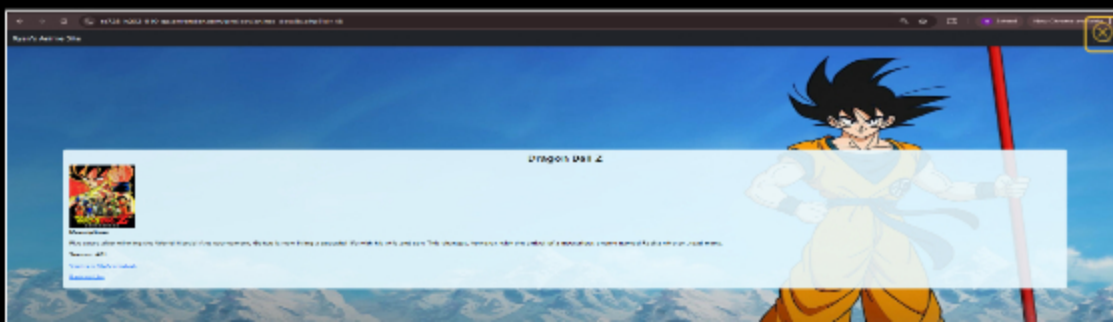
- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



this shows the admin with an api with the edit and delete on the bottom



this shows the admin with an manual



this shows the same api as before but without the admin role

 Saved: 5/3/2026 7:37:12 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL#1

https://rc728-it202-010-prod.onrender.com/project/anime_details.php?id=48



URL

[https://rc728-it202-010-prod.](https://rc728-it202-010-prod.onrender.com/project/anime_details.php?id=48)



URL#2

<https://github.com/524RC/RC728-IT202-010>



URL

[https://github.com/524RC/R](https://github.com/524RC/RC728-IT202-010)



 Saved: 5/3/2026 4:42:47 AM

Section #5: (150 pts.) Edit Page

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/ missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied

```
$id = (int)se($_GET, "id", 0, false);  
if ($id <= 0) {  
    flash("Invalid ID", "danger");  
    die(header("Location: " . get_url("list_anime.php")));  
}  
$db = getDB();  
$stmt = $db->prepare("SELECT * FROM Anime WHERE id = :id");  
$stmt->execute();  
$anime = $stmt->fetch();
```

```

$stmt->execute(["id" => $id]);
$anime = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$anime) {
    flash("Anime not found", "danger");
    die(header("location: " . get_url("list_anime.php")));
}

```

this shows the code that handles the loading and invalid id handling

```

<?php
// Anime Controller
class AnimeController {
    public function __construct() {
        $this->pdo = new PDO("mysql:host=localhost;dbname=anime_db", "root", "root");
        $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    }

    public function get_anime($id) {
        $stmt = $this->pdo->prepare("SELECT * FROM anime WHERE id = ?");
        $stmt->execute([$id]);
        $anime = $stmt->fetch(PDO::FETCH_ASSOC);

        if (!$anime) {
            return null;
        }

        return $anime;
    }

    public function create_anime($title, $year, $studio, $genres) {
        $stmt = $this->pdo->prepare("INSERT INTO anime (title, year, studio, genres) VALUES (?, ?, ?, ?)");
        $stmt->execute([$title, $year, $studio, $genres]);

        return true;
    }

    public function update_anime($id, $title, $year, $studio, $genres) {
        $stmt = $this->pdo->prepare("UPDATE anime SET title = ?, year = ?, studio = ?, genres = ? WHERE id = ?");
        $stmt->execute([$title, $year, $studio, $genres, $id]);

        return true;
    }

    public function delete_anime($id) {
        $stmt = $this->pdo->prepare("DELETE FROM anime WHERE id = ?");
        $stmt->execute([$id]);

        return true;
    }
}

```

php validation

```

// Anime Validation
class AnimeValidator {
    public function validate($data) {
        $errors = [];

        // Title validation
        if (empty($data['title'])) {
            $errors['title'] = "Title is required";
        }

        // Year validation
        if (!is_numeric($data['year']) || $data['year'] < 1900 || $data['year'] > 2024) {
            $errors['year'] = "Year must be a valid year between 1900 and 2024";
        }

        // Studio validation
        if (empty($data['studio'])) {
            $errors['studio'] = "Studio is required";
        }

        // Genres validation
        if (empty($data['genres'])) {
            $errors['genres'] = "Genres are required";
        }

        return $errors;
    }
}

```

this shows the js validation

```

// Anime Form Validation
class AnimeFormValidator {
    public function validate($data) {
        $errors = [];

        // Title validation
        if (empty($data['title'])) {
            $errors['title'] = "Title is required";
        }

        // Year validation
        if (!is_numeric($data['year']) || $data['year'] < 1900 || $data['year'] > 2024) {
            $errors['year'] = "Year must be a valid year between 1900 and 2024";
        }

        // Studio validation
        if (empty($data['studio'])) {
            $errors['studio'] = "Studio is required";
        }

        // Genres validation
        if (empty($data['genres'])) {
            $errors['genres'] = "Genres are required";
        }

        return $errors;
    }
}

```

html validation and the code that generates the form with the correct data types and populates it

```

<div class="anime-box">
    <div class="anime-form">
        <input type="text" value="" />
        <input type="text" value="" />
        <input type="text" value="" />
        <input type="text" value="" />
    </div>
</div>

```

this shows the styling

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the form is generated and submission is handled
- Briefly note the styling choices
- Note what fields are editable and why the choice was made for them (it's likely not all fields are editable, especially `id`, `created`, `modified`, etc)

Your Response:

Loading logic: the loading logic works by the page retrieving the anime ID from the url using `$_GET` and converts it into an integer. It then checks if the ID is valid and if it is then the anime data is fetched from the database and stored in variables to be used in the form. If the ID is invalid or the anime was not found, then the user is redirected back to the list page with an error message. This is very similar to how the view page works.

Generated and submission: the form is generated by using the HTML inputs like `type="url"` to ensure that a valid url is being used along with other validation attributes like `required` and `maxlength`. The form fields are pre-populated using PHP variables so the user can see and edit the existing data. Right before the submitted, a JS validation is ran to check the inputs. After the JS validation, the php validation is ran on the server to ensure that the data is valid before updating the query to save the changes done to the database.

Styling: The styling choice i did for the edit was similar to the rest using the same `.anime_box` class to make the background semi-transparent along with padding, margin, and border-radius to make the spacing better and making the form look more clean and readable.

editable fields: the fields that are editable are the title, description, the `picture_url`, and the `mal_url` because these are the properties that user can see which may require updates. The fields like `id`, `created`, and `modified` are not editable because they are managed by the database and they shouldn't be altered to maintain data integrity.

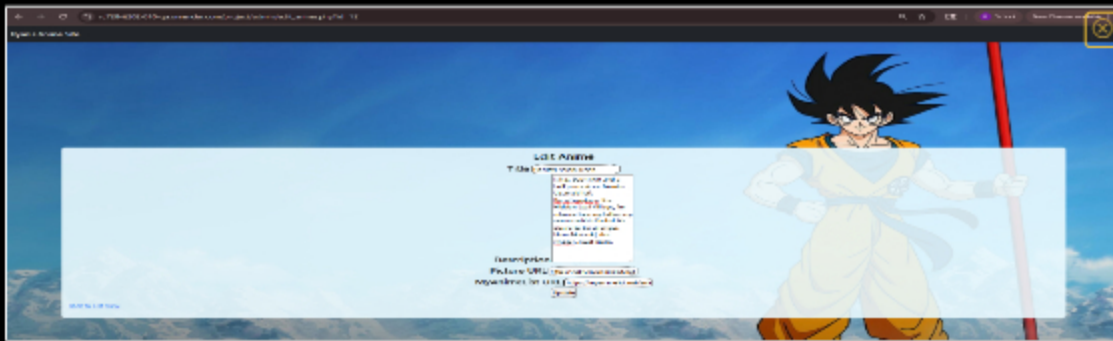
 Saved: 5/3/2026 5:38:17 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

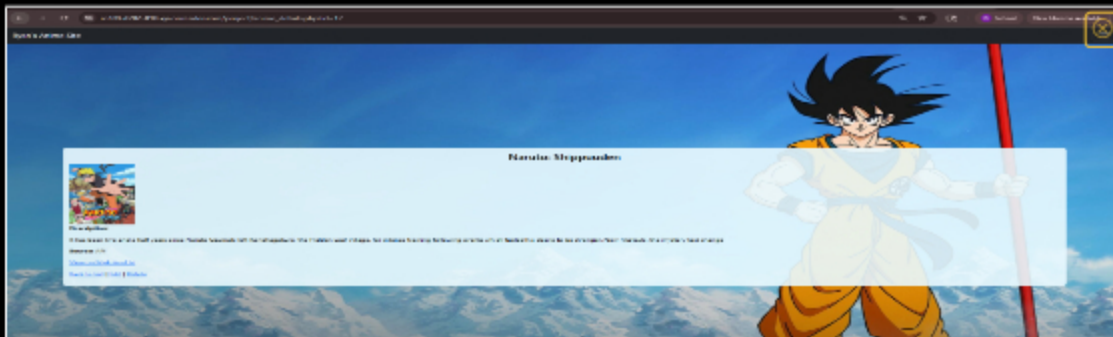
- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)



before update



after update



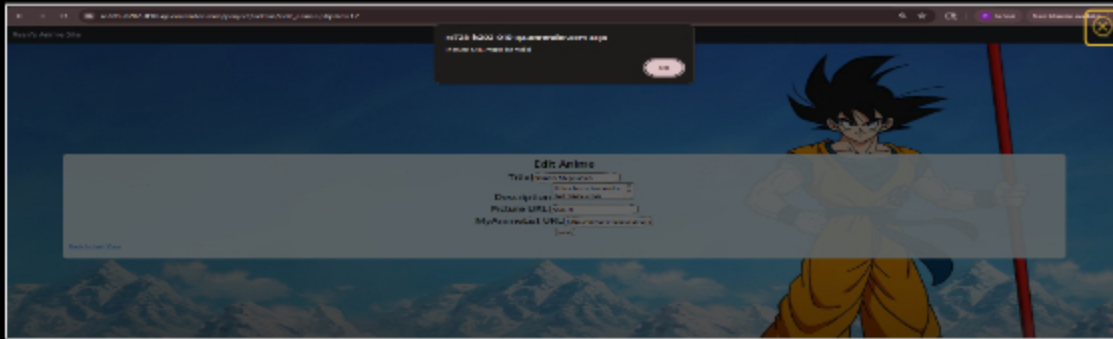
update is shown in view



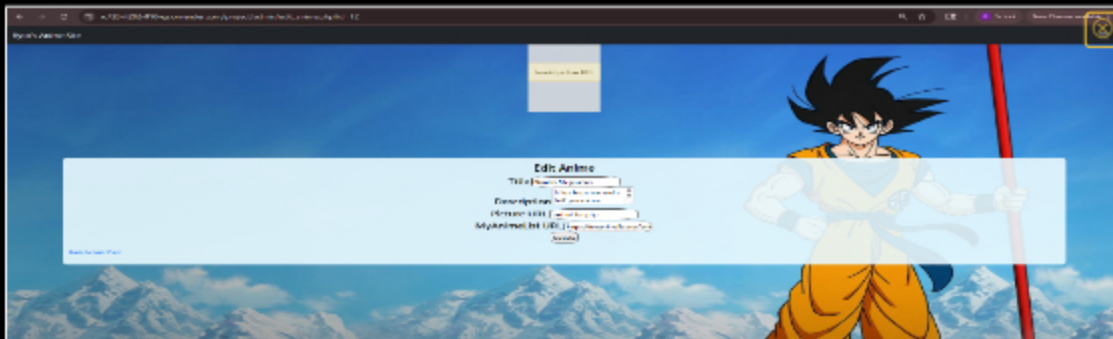
html validation



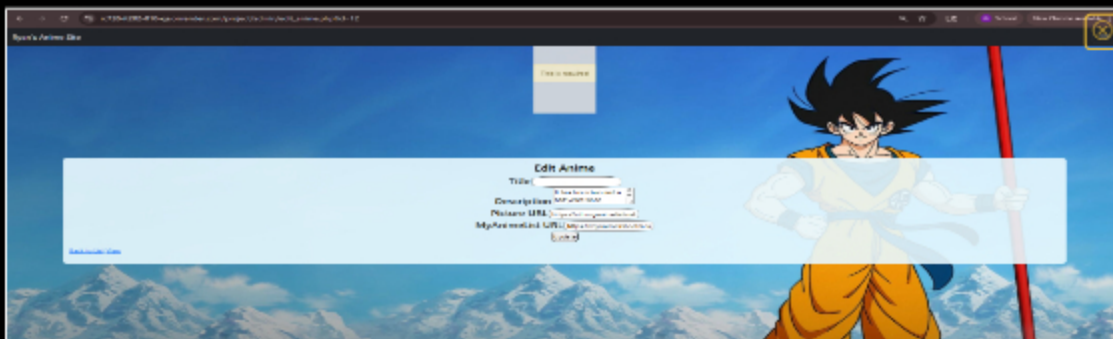
js validation title error



js url error



php url error



php title error

 Saved: 5/3/2026 5:59:24 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL#1

https://rc728-it202-010-prod.onrender.com/project/admin/edit_anime.php?id=12



https://rc728-it202-010-prod.



URL#2

<https://github.com/524RC/RC728-IT202-010>



https://github.com/524RC/R



URL#3

<https://github.com/524RC/RC728-IT202-010>



https://github.com/524RC/R



Saved: 5/3/2026 6:00:28 PM

Section #6: (150 pts.) Delete Logic

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/ sort criteria

```
if (!has_role("Admin")) {  
    flash("No permission", "danger");  
    die(header("Location: " . get_url("landing.php")));  
}
```

handles the role permission

```
$id = (int)se($_GET, "id", 0, false);  
$return = se($_GET, "return", get_url("list_anime.php"), false);  
  
if ($id <= 0) {
```

```
flash("Invalid ID", "danger");
die(header("Location: " . $return));
}
```

ID retrieval and invalid id handling

```
$stmt = $db->prepare("DELETE FROM Anime WHERE id = :id");
try {
    $stmt->execute(["id" => $id]);
    flash("Anime deleted", "success");
} catch (PDOException $e) {
    flash("Error deleting anime", "danger");
}
die(header("Location: " . $return));
```

deletion logic and the redirection logic that goes back to the previous page

```
<a
    href="<?php echo get_url('admin/delete_anime.php?id=' . $anime['id'] . '&return=' . urlencode($_SERVER['REQUEST_URI'])); >"
    onclick="return confirm('Are you sure you want to delete this anime?');"
>
    Delete
</a>
```

maintains filter/sort



Saved: 5/3/2026 7:04:38 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/ rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/ sort data

Your Response:

How it works: The deletion logic works by the page getting the anime ID from the url using `$_GET` and converts it to an integer. This checks if the ID is valid and if it is, then an SQL statement is used to delete the record from the DB. If the id is not valid, then it redirects the user back to the previous page with an error message. There is a successful deletion message or an error message based off of the results.

soft or hard delete: this uses a hard delete because the record is permanently removed the DB using a DELETE statement. There isn't a flag or status that marks the record as deleted, it just completely wipes the record off the DB.

Permissions: the only users that can delete records is users with the admin role. If a user without the admin role tries to access the delete page, they are then redirected and a message is shown that they have no permission to access that page.

Redirect and maintaining filters/ sort: The delete link has a return parameter in it that stores the current page URL by using `$_SERVER['REQUEST_URI']`. After deleting, the page redirects back to the URL that is stored which also keeps any of the filters, search terms, or sorting that was applied on the list page.

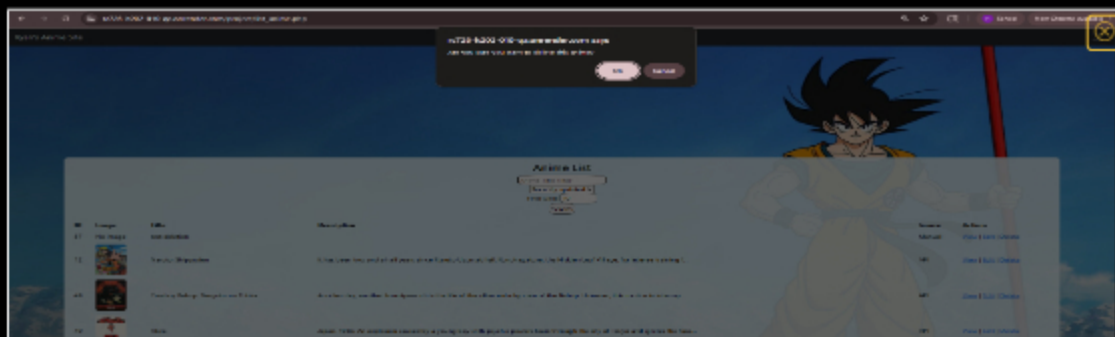
 Saved: 5/3/2026 7:04:38 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

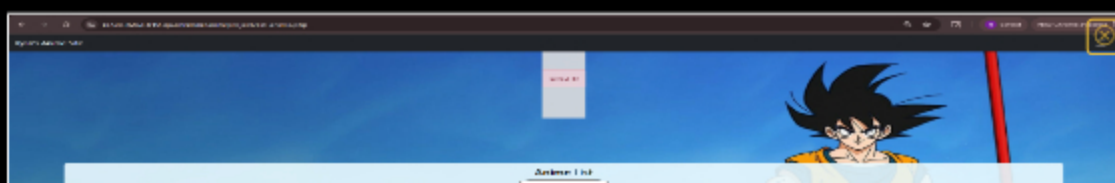
- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown



deletion confirmation in background you can see test deletion

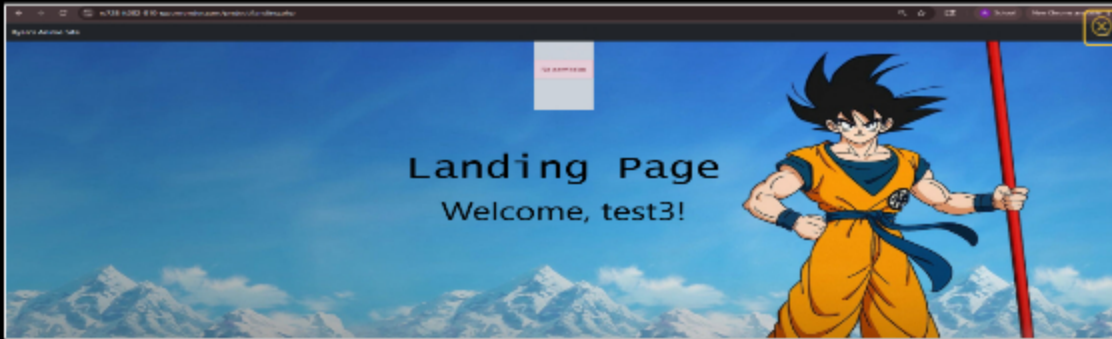


anime deletion, you can see test deletion is no longer there





invalid id error



no permission error



Saved: 5/3/2026 7:18:17 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL#1

https://rc728-it202-010-prod.onrender.com/project/admin/delete_anime.php?id=12



URL

https://rc728-it202-010-prod.onrender.com/project/admin/delete_anime.php?id=12



URL#2

<https://github.com/524RC/RC728-IT202-010/pull/4>



URL

<https://github.com/524RC/RC728-IT202-010/pull/4>



Saved: 5/3/2026 7:23:40 PM

Section #7: (50 pts.) Misc

Progress: 100%

Task #1 (16.67 pts.) - Github Details

Progress: 100%

Part 1:

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)

commits tab

commits tab

Saved: 5/3/2026 7:26:28 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`) (Milestone2 into qa)

URL#1

<https://github.com/524RC/RC728-IT20201104>



UT <https://github.com/524RC/RC728>



URL#2

<https://github.com/524RC/RC728-IT20201108>

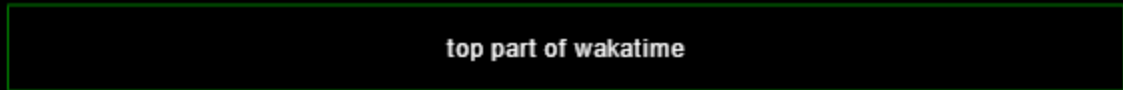


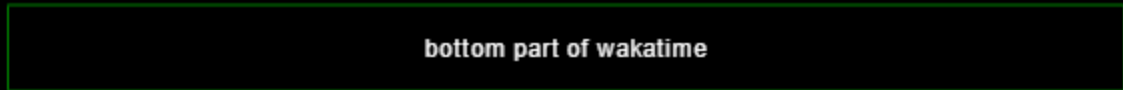
UT <https://github.com/524RC/RC728>



Saved: 5/3/2026 7:26:28 PM

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary





≡ Task #3 (16.67 pts.) - Reflection

⇒ Task #1 (0.33 pts.) - What did you learn?

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to delete records form the DB and how more about bootstrap. I

found it quite interesting a learned about soft and hard delete and more things that bootstrap can do.



Saved: 5/3/2026 7:31:12 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the this assignment was the styling of each page. I quite enjoy styling the website and it wasn't that hard to work around the css. A lot of the css code can be found on w3schools.



Saved: 5/3/2026 7:30:30 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part about this assignment I would say was a little bit of deletion and trying to get it to work and the other hard part was the bootstrap and trying to get it to work for my website. The bootstrap was rather difficult to manage cause I grab a code from the bootstrap website and I had to try and make it work for my website along with buttons not working or sizing being off.



Saved: 5/3/2026 7:33:43 PM